

Lecture notes

Lecture 3: Query Execution

Sofija Kochovska - 89252112

Contents

1. Introduction.....	2
2. CPU Architecture Basics: Why DBMSs Stumble	2
2.1 DBMS/CPU PROBLEMS	3
2.1.1 Case Study: Selection Scans	3
3. Query Execution	4
4. Processing Models	5
4.1 Iterator Model	5
4.2 Materialization model	6
4.3 Vectorized/Batch Model (“Fragmentation Model”).....	6
5. Plan Processing Direction: Pull vs. Push	7
5.1 Filter Representation	7
6. Parallel Execution: Harnessing Multiple Cores	8
6.1 Exchange Operators	8
7. Summary Table	9
8. Conclusions.....	10
9. References.....	11

1. Introduction

The central theme is that DBMS engineering is an orchestration of optimizations to fully utilize modern hardware, with no single technique being overwhelmingly more important than others.

The story of modern execution optimization truly gained momentum with the work on MonetDB/X100 around 2005. A research group in the Netherlands conducted a low-level analysis of execution bottlenecks in in-memory DBMSs for OLAP workloads. Their key finding was that contemporary DBMSs were designed incorrectly for modern CPU architectures. They identified critical CPU/DBMS interaction problems and, based on these findings, built a new DBMS called MonetDB/X100 (later known as Vectorwise, Actian Vector, and Avalanche) to demonstrate their solutions. This work laid the foundation for many of the concepts discussed in this lecture.

The primary optimization goals are:

- **Reduce Instruction Count:** Do the same work with fewer instructions. This is critically important because at the scale of data processing in DBMSs, even a single saved instruction per data item can lead to massive time savings. For example, consider processing 1 GB of data, which is roughly 100 million tuples (at ~10 bytes/tuple). If you can eliminate just one instruction per tuple, and that instruction takes 60 nanoseconds to execute, the total time saved is: $100,000,000 \text{ tuples} * 60 \text{ ns/instruction} = 6,000,000,000 \text{ ns} = 6 \text{ seconds}$. This demonstrates how microscopic savings per operation compound into significant performance gains at the database scale.
- **Reduce Cycles per Instruction (CPI):** Execute the necessary instructions in fewer CPU cycles. This involves optimizing for the CPU's internal architecture, such as its pipeline, cache hierarchy, and branch prediction units.
- **Parallelize Execution:** Leverage multiple CPU cores and threads to compute different parts of a query simultaneously.

2. CPU Architecture Basics: Why DBMSs Stumble

To understand the problems identified by the MonetDB team, we must first understand modern CPU fundamentals.

- **Pipelining:** CPUs break down instruction execution into discrete stages (e.g., fetch, decode, execute, memory access, write-back). The goal is to keep all stages busy every cycle, like an assembly line. While one instruction is being executed, the next is being decoded, and the one after that is being fetched.
- **Super-scalar and Out-of-Order Execution:** Modern CPUs are super-scalar, meaning they have multiple pipelines. They can execute several instructions in a single cycle, provided the instructions are independent. Out-of-order execution allows the CPU to dynamically

reorder independent instructions to keep its pipelines full, masking delays from slow operations like memory accesses.

2.1 DBMS/CPU PROBLEMS

However, DBMSs often disrupt this finely tuned machinery, leading to two major problems:

- **Problem #1: Data Dependencies:** If one instruction requires the result of a previous instruction that hasn't finished, the dependent instruction cannot be pushed into the pipeline. The CPU must stall, waiting for the dependency to be resolved, leaving pipeline stages idle. DBMS code, with its chains of operations on tuple data, is often full of such dependencies.
- **Problem #2: Branch Misprediction:** To keep its pipeline full, the CPU must guess which way a branch (e.g., an if or while condition) will go before the condition is actually evaluated. It speculatively executes the predicted path. If the prediction is correct, performance is excellent. If it's wrong, the CPU must flush the entire pipeline of all speculatively executed work and start over. This is extremely costly. The most frequently executed branching code in a DBMS is the filter operation during a sequential scan. Since the selectivity of a filter is data-dependent and random, it is nearly impossible for the CPU to predict correctly, leading to a high rate of pipeline flushes and severe performance penalties.

2.1.1 Case Study: Selection Scans

A Selection Scan (often called a sequential scan with a predicate or simply a WHERE clause execution) is one of the most fundamental and frequently executed operations in a DBMS. Its purpose is to iterate through every tuple in a table (or a segment of a table) and return only those that satisfy a specified condition. The core challenge in optimizing a selection scan lies in the evaluation of the filter condition (e.g., WHERE key > low AND key < high). How this condition is checked inside the tight loop of the scan has profound implications for CPU performance.

Two primary implementations are:

Scalar (Branching): The Naive Approach

```
# Pseudocode for Branching Selection Scan
i = 0
for each tuple in table:
    key = get_key(tuple)
    if (key > low) and (key < high): # This is a BRANCH
        copy(tuple, output[i])
        i = i + 1
```

Problem: The if condition is a branch in the CPU's execution path. For each tuple, the CPU must predict whether the branch will be taken

Scalar (Branchless): The Optimized Approach

```
# Pseudocode for Branchless Selection Scan
i = 0
for each tuple in table:
    copy(tuple, output[i]) # Always copy the tuple
    tentatively
    key = get_key(tuple)
    # Use conditional logic to create a mask (0 or 1)
    mask = (key > low) ? 1 : 0 # These are conditional
    mask = mask & ((key < high) ? 1 : 0) # operations,
    NOT branches.
    i = i + mask # Only advance i if the tuple is valid
```

(the condition is true) or not taken (false). It then speculatively executes the predicted path to keep its pipeline full.

Why it fails in DBMSs: The outcome of this branch is essentially random, depending entirely on the data distribution and the selectivity of the filter. The CPU's prediction logic (e.g., "predict the same as last time") fails miserably with random data. When the prediction is wrong - which is about 50% of the time for a 50% selective filter - the CPU must flush its pipeline, discarding all the speculative work. This is a massive penalty, costing 10-20 cycles per misprediction. The provided graph shows this cost clearly: the "branching" line has high and variable CPU cycles per tuple, peaking where selectivity is around 50% (the hardest-to-predict scenario).

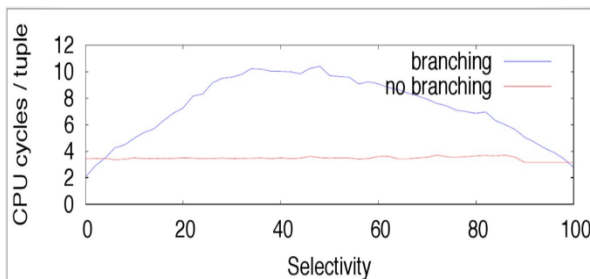


Figure 1. Branching versus branchless performance

This clever approach rewrites the logic to avoid the branch altogether, using conditional operations (like conditional moves or bitwise arithmetic) that the CPU can execute without speculation.

How it works: The tuple is always copied into the output buffer at position i . The key is then checked, and the condition is evaluated to a numeric value (delta or mask of 0 or 1). The index i is then incremented by this value. If the condition is false ($mask=0$), the index doesn't change, and the next tuple will overwrite the invalid one. If true ($mask=1$), the index advances, preserving the valid tuple.

Why it's better: There is no branch for the CPU to mispredict. The control flow is a single, predictable loop. The CPU's pipeline remains full and never needs to be flushed. This results in a consistent, low number of CPU cycles per tuple, as shown by the flat "no branching" line on the graph. While it may do a tiny bit more work (an unconditional copy), this cost is far less than the penalty of a single branch misprediction.

3. Query Execution

Query execution is the process of carrying out a query plan generated by the query optimizer. It defines how data will be accessed, processed, and combined to produce the final query result. Execution is typically modeled as a set of operators organized in a Directed Acyclic Graph (DAG), where each node performs a specific operation on data.

1. Query Plan

- a. A query plan represents the step-by-step data processing strategy for a query.
- b. It is structured as a DAG of operators, where:
 - i. Nodes are the operators (e.g., scan, join, filter, aggregate).
 - ii. Edges represent the flow of intermediate data between operators.

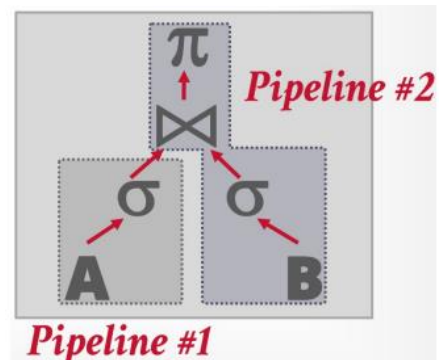


Figure 2. Query plan

- c. The plan defines both logical operations (what to do) and physical operations (how to do it efficiently).
- 2. Operator instance**
 - a. An operator instance is a *specific execution* of an operator on a **unique segment of data**.
 - b. Example: If a dataset is divided into multiple partitions, the same operator (e.g., a scan) may run separately on each partition as distinct operator instances.
 - c. This supports parallelism, allowing multiple instances to execute simultaneously.
- 3. Task**
 - a. A **task** is a *sequence of one or more operator instances* executed together as a single unit of work.
 - b. Each task typically runs in one execution thread or process.
 - c. Tasks are the fundamental scheduling units in distributed query execution systems.
- 4. Task Set**
 - a. A **task set** is the *collection of all executable tasks* that implement a **logical pipeline** of the query plan.
 - b. The system schedules and runs these tasks (potentially in parallel) to complete the query efficiently.
 - c. Task sets allow scalable and distributed query processing.

4. Processing Models

A DBMS's processing model defines the system's high-level strategy for executing a query plan. It dictates how control moves between operators and how data flows from one operator to the next. The choice of model involves trade-offs, making different models better suited for OLTP (numerous, simple queries) versus OLAP (complex, data-intensive queries) workloads.

4.1 Iterator Model

This is the most common model, used by systems like PostgreSQL, MySQL, and SQLite.

- **Mechanism:** Every operator implements a `Next()` method. Each call to `Next()` returns a **single tuple**. The root operator calls `Next()` on its child, which calls `Next()` on its child, and so on, pulling data up from the base tables one tuple at a time. Operators also have `Open()` and `Close()` methods for initialization and cleanup.
- **Control Flow:** Pull-based, top-to-bottom.
- **Data Flow:** Typically, whole tuples (NSM - N-ary Storage Model) are passed between operators.
- **Advantages:**
 - **Simplicity:** Intuitive to implement and debug.
 - **Pipelining:** Allows for efficient processing where a tuple can flow through multiple operators (e.g., Project → Filter → Scan) without being materialized in between. This is a form of inter-operator parallelism.
- **Disadvantages:**

- **High Overhead:** A single query can result in millions of virtual function calls (Next()), each with a cost.
- **Poor CPU Utilization:** The per-tuple control flow makes it difficult for the CPU to predict branches and optimize instruction caching.

4.2 Materialization model

This model takes a very different approach, used by systems like MonetDB.

- **Mechanism:** Each operator processes **its entire input all at once** and materializes its **complete output** before passing it to the parent operator. An operator's method might be called Output() instead of Next().
- **Control Flow:** Can be pull or push, but processing is atomic per operator.
- **Data Flow:** Can be whole tuples or, as in MonetDB's case, entire columns (DSM - Decomposition Storage Model).
- **Advantages:**
 - **Low Overhead:** Excellent for OLTP workloads where queries only access a small number of tuples. The number of function calls is minimal.
 - **Direct Access:** Parent operators can efficiently scan materialized results.
- **Disadvantages:**
 - **Inefficient for OLAP:** Can create huge intermediate results that must be spooled to disk, consuming memory and I/O bandwidth. The "pipeline breakers" are inherent to the model.

4.3 Vectorized/Batch Model (“Fragmentation Model”)

This model, inspired by the MonetDB/X100 work, strikes a balance and is ideal for OLAP. It's used by systems like Vectorwise, Snowflake, and Amazon Redshift.

- **Mechanism:** A hybrid where each operator implements a Next() method, but it returns a **batch of tuples** (a vector), not a single one. The internal loops of operators are then designed to process this entire batch in a tight loop.
- **Control Flow:** Pull-based, top-to-bottom (like Iterator model).
- **Data Flow:** Batches of tuples, which can be row-based or columnar.
- **Advantages:**
 - **Reduced Overhead:** Drastically reduces the number of function calls compared to the Iterator model.
 - **Superior CPU Utilization:**
 - **Compiler Optimization:** Tight loops over arrays allow compilers to auto-vectorize code, using SIMD (Single Instruction, Multiple Data) instructions to process multiple tuples in a single CPU cycle.
 - **Cache Efficiency:** Data is accessed in contiguous blocks, which is cache-friendly.

- **Branch Prediction:** While branches still exist, their cost is amortized over a batch of tuples. The CPU can predict the branch outcome for the entire batch more effectively, or the DBMS can use branchless techniques within the batch.

5. Plan Processing Direction: Pull vs. Push

This is an orthogonal concept to the processing model, dealing with the initiation of query execution.

- **Top-to-Bottom (Pull):** This is the classic approach, aligned with the Iterator Model. Execution starts at the root of the query plan tree, which "pulls" data up by calling Next() on its children.
 - **Pros:** Natural output control (e.g., LIMIT can stop pulling at any time).
 - **Cons:** The constant virtual function calls for Next() incur a performance overhead.
- **Bottom-to-Top (Push):** Execution is driven from the leaves. A scheduler takes over, reading data from base tables and "pushing" it up through the operator tree. Systems like HyPer and DuckDB use this.
 - **Pros:** Can be significantly faster. It allows for "fusing" operators - compiling a segment of the query plan (e.g., Scan → Filter → Project) into a single, tight, branchless machine code loop, eliminating interpreter overhead entirely.
 - **Cons:** More complex to implement, and it can be harder to control the size of intermediate results or implement operators that require seeing all their input first (e.g., Sort-Merge Join).

5.1 Filter Representation

When vectorized execution is used, not all tuples in a batch may satisfy a filter condition. To manage this efficiently, DBMSs use two representations:

1. **Selection Vectors**
 - A dense list of tuple identifiers indicating valid tuples.
 - Preallocated to the maximum input batch size.
 - Easy to iterate but may require extra indirection.
2. **Bitmaps**
 - A positional bitmap where each bit represents a tuple's validity.
 - Efficient for SIMD processing since many CPUs natively handle bitmask operations.

6. Parallel Execution: Harnessing Multiple Cores

A single-threaded execution engine cannot utilize modern multi-core servers. DBMSs employ two main types of parallelism:

- **Inter-Query Parallelism:** The simplest form. The DBMS executes multiple different queries simultaneously on different CPU cores. This improves overall system throughput.
- **Intra-Query Parallelism:** Speeds up a single, complex query by breaking its work down across multiple cores. This is essential for OLAP.
 1. **Intra-Operator (Horizontal Parallelism):** Also known as "partition parallelism." The same operator is executed by multiple threads, each working on a different partition of the data. For example, four threads can simultaneously scan different parts of a table. The results are combined using an **Exchange Operator**, which acts as a synchronization point in the plan. There are three types of exchange:
 - **Gather:** Merges results from multiple parallel workers into a single stream.
 - **Distribute:** Splits a single stream into multiple output streams to feed parallel workers.
 - **Repartition:** Re-shuffles multiple input streams to multiple outputs, essential for parallel joins and aggregations (e.g., HASH REPARTITION).
 2. **Inter-Operator (Vertical / Pipelined Parallelism):** Different operators within a query plan are executed concurrently. While one operator is consuming tuples, the next operator in the pipeline is processing the tuples it has already produced. This is naturally supported by the Iterator model but requires careful scheduling to avoid stalls.

6.1 Exchange Operators

Exchange operators are **special system operators** that manage communication between parallel threads or processes. They are inserted into query plans to control how data flows among concurrent workers.

There are **three types of exchange operators**, each serving a unique coordination role:

1. **Gather Exchange**
 - **Purpose:** Combines results from multiple parallel workers into a single output stream.
 - **Example:** After several threads finish scanning partitions of a table, their results are gathered and sent to a single aggregation operator.
2. **Distribute Exchange**
 - **Purpose:** Splits a single input stream into multiple output streams to feed parallel workers.
 - **Example:** A scan operator may distribute its output to several join operators working in parallel.
3. **Repartition Exchange**

- **Purpose:** Redistributes (shuffles) multiple input streams into multiple output streams based on a partitioning key.
- **Use Case:** Essential for **parallel joins and aggregations**, where tuples must be grouped by key (e.g., HASH REPARTITION).
- **Example:** In a distributed parallel hash join, each worker sends tuples with the same join key to the same destination thread.

These exchange points are **critical synchronization boundaries**. They help merge, split, or redistribute data between pipeline segments, enabling scalable and balanced execution across multiple cores.

7. Summary Table

Concept / Approach	Description	Best For	Key Characteristics
Execution Optimization	Techniques to exploit hardware capabilities through efficient code and parallelism	General DBMS performance	Reduces instruction count, cycles per instruction, or parallelizes execution
Iterator Model	Tuple-by-tuple operator execution via Next() calls	OLTP, mixed	Simple, pipelined, but high function-call overhead
Materialization Model	Processes full input sets and materializes results	OLTP	Low overhead, avoids per-tuple calls, poor for large OLAP queries
Vectorized Model	Operators process batches of tuples using SIMD	OLAP	Efficient use of CPU pipelines and caches, minimal branching
Top-Down (Pull) Execution	Root operator pulls tuples from children	General workloads	Easy to implement, supports LIMIT, but less cache-efficient
Bottom-Up (Push) Execution	Leaf operators push tuples upward	Analytical workloads	Fewer function calls, optimized for caches and tight loops
Filter Representation (Selection Vector)	Dense list of valid tuple IDs within a batch	Vectorized systems	Efficient for compact filtering; pre-allocated
Filter Representation (Bitmaps)	Bitmap marks valid tuples; compatible with SIMD	Vectorized systems	Hardware-optimized masking, fast filtering

Inter-Query Parallelism	Runs multiple different queries simultaneously	Mixed workloads	Improves system throughput, manages concurrency
Intra-Query Parallelism	Parallelizes a single complex query	OLAP	Includes intra- and inter-operator techniques
Intra-Operator (Horizontal) Parallelism	Same operator runs on partitions of data	Large scans, joins	Uses all cores for a single operator; requires exchange operators
Inter-Operator (Vertical) Parallelism	Different operators execute concurrently in a pipeline	Long query plans	Improves overlap of stages; requires scheduling
Exchange Operators (Gather / Distribute / Repartition)	Synchronize data among threads in parallel plans	Parallel DBMS execution	Combine, split, or shuffle data across workers
Pipeline Breakers	Operators that block until input is complete (e.g., Sort, Join build)	OLAP	Force materialization between stages
CPU Pipelining & Branch Prediction	Hardware-level performance factors	Low-level optimization	Affects instruction scheduling and branch handling
Code Specialization (JIT / Precompilation)	Dynamically compiles query-specific code	High-performance DBMS	Eliminates interpretation overhead, improves CPU efficiency
Vectorized Filters and SIMD	Process multiple values per instruction	OLAP	Exploits modern CPU features for bulk data operations

8. Conclusions

The evolution of query execution engines reflects the broader trend in computing - adapting software to exploit the full potential of modern hardware. Early systems relied on the simple iterator model, which, while intuitive, struggled with CPU inefficiencies such as branching and poor cache locality. Research efforts like MonetDB/X100 transformed our understanding by showing how vectorized processing, SIMD instructions, and cache-conscious algorithms could drastically improve analytical performance.

Modern DBMSs combine multiple execution paradigms, such as push-based pipelines, JIT compilation, and intra-query parallelism, to achieve both high throughput and scalability. Systems like **HyPer**, **DuckDB**, and **Snowflake** exemplify this new generation, blending traditional relational principles with aggressive low-level optimizations.

In summary:

- **OLTP systems** benefit from lightweight materialization and simple control flow.
- **OLAP systems** thrive on vectorization, parallel pipelines, and bottom-up execution.
- Future engines will continue to merge **compiler technology** with **database architecture**, producing query execution strategies that are dynamically optimized for both hardware and workload.

9. References

Pavlo, A. (2025). Advanced Database Systems Lecture Notes: Query Execution and Processing. Carnegie Mellon University. Based on materials used in the CMU Advanced Database Systems course.

Boncz, P. A., et al. (2005). MonetDB/X100: Hyper-Pipelining Query Execution. Proceedings of the Conference on Innovative Data Systems Research (CIDR).

Selinger, P. G., Astrahan, M. M., et al. (1979). Access Path Selection in a Relational Database Management System. Proceedings of the ACM SIGMOD Conference on Management of Data.

Kemper, A., & Neumann, T. (2011). HyPer: A Hybrid OLTP and OLAP Main Memory Database System Based on Virtual Memory Snapshots. Proceedings of the IEEE International Conference on Data Engineering (ICDE).